Thetawave.ai

💻 轉接器模式 – 相容性問題與系統整合

Generated by Thetawave.ai

🧩 轉接器模式 (Adapter Pattern) 概念與應用

> **轉接器模式** 是一種結構型設計模式，用於「讓一個介面相容於另一個介面」，解決不同介面間不兼容的問題，使原本不相容的類別能夠合作。

---

1. Client、介面與相容性問題

- **Client** 只能操作類別公開的介面 (Interface)，只能看到類別的「表面」。
- 介面是「介於 Client 與類別之間的一面」(A Face In Between Two Things)。
- 不同模組或第三方套件可能有不同介面，Client 定義的介面與實際可用的類別不相容，造成整合困難。

---

2. 案例背景：英文單字查詢程式

角色	職責說明
Client (使用者互動介面)	透過命令列介面要求查詢英文單字，輸入單字拼寫，顯示單字在各詞性下的定義。
小華 (爬蟲開發者)	負責開發從網路爬取單字定義的爬蟲程式 (Super Vocab Crawler)。

- Client 端定義了查詢單字的介面 VocabDictionary 及其操作 lookup。

- 小華開發的爬蟲類別 Super Vocab Crawler 不符合 Client 定義的介面，且不允許修改其程式碼。

---

3. 物件導向分析重點

概念	說明
Word	代表單字，有拼寫 (Spelling) 及多詞性(Part of Speech)下的定義。
Part of Speech (詞性)	以 Enum 列舉八大詞性，每個詞性有對應大寫字母符號表示。
Definitions (定義)	以 Map 結構實作，Key 為詞性，Value 為該詞性下的多重定義列表。
Qualifier (限定符)	UML 關聯中的 Key，用以表示 Key-Value 關聯的 Key 端詞性。

- 使用 Map (雜湊資料結構)實作 Word 與 Definitions 的關聯。
- 定義 VocabDictionary 介面，抽象查詢單字的意圖。
- Client 端 VocabLookupCommonLanguageInterface 依賴此介面與使用者互動。

---

4. 相容性問題與三道「Force」

編號	說明
第一道 Force	Client 依賴自己定義的介面 (Target 介面)。
第二道 Force	實際實作的類別介面與 Client 定義不相容，且不可修改。
第三道 Force	希望重複利用實作者的能力，但不影響 Client 原先介面。

- **相容性 (Compatibility)** 是第二道 Force 的核心。
- 問題核心：如何利用另一介面的能力去實作 Client 所定義的介面。

Thetawave.ai

5. 轉接器模式解決方案

元件	角色與功能
Target 介面	Client 定義的介面，表達其意圖 (例：VocabDictionary)。
Adapter 類別	實作 Target 介面，持有另一介面物件，將請求轉接給該物件。
Adaptee 介面	實際有能力提供服務的類別介面 (例：Super Vocab Crawler)。

- **Adapter** 實作 Target 介面，並持有 Adaptee 物件。
- 在 Adapter 的方法中，委派 Adaptee 的功能並轉換回 Client 預期的型態與例外。

---

6. 轉接器模式實作細節與挑戰

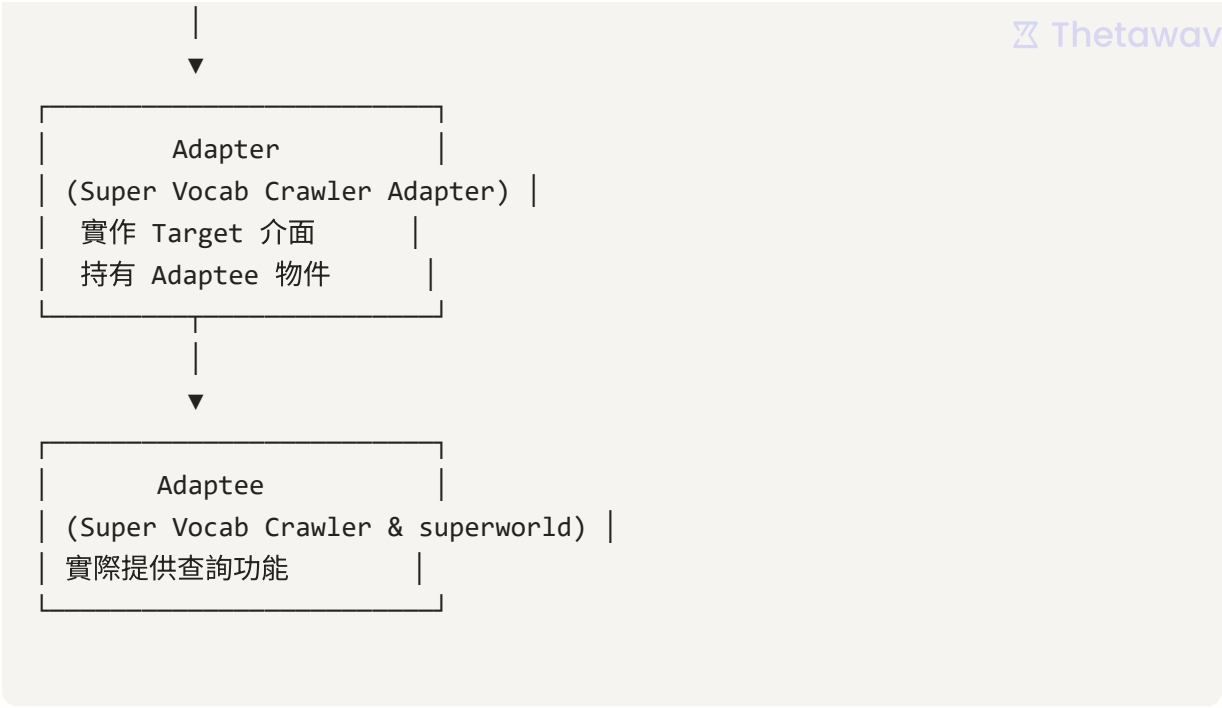
挑戰	解決方法
例外型態不一致	Adapter 中捕捉 Adaptee 拋出的例外，轉成 Client 預期的例外型態。
資料格式不一致	Adapter 內部轉換 Adaptee 回傳的資料型態（如 superworld 轉 Word）。
不可修改第三方類別	Adapter 將第三方類別包裝起來，維護原介面一致性與 Client 隱藏實作。

---

7. 轉接器模式的程式碼架構示意

```
┌──────────────────────────┐
│         Client           │
│ (VocabLookupCommonLanguageInterface) │
│    依賴 Target 介面      │
└──────────┬───────────────┘
```

Thetawave.ai



---

8. 轉接器模式的優點與缺點

項目	說明
優點	- Client 不需改變，維持原有介面。 - 可整合第三方類別或不相容介面。 - 封裝轉接細節，不污染其他程式碼。
缺點	- Adapter 類別需維護，當介面改動需同步調整。

---

9. 依賴反轉原則 (Dependency Inversion Principle, DIP)

> 「高層模組不應依賴低層模組，兩者皆應依賴抽象。」

- **模組**：多個類別或介面組成的集合，依據意圖層次可分高層與低層。
- **高層模組**：較接近需求意圖的模組 (如 Client 介面)。
- **低層模組**：較靠近實作細節的模組 (如爬蟲執行模組)。

DIP 實踐步驟

Thetawave.ai

步驟	說明
1	在高層模組中定義抽象介面，表達意圖，依賴於此介面。
2	低層模組實作該抽象介面，讓高層模組不直接依賴低層實作。

- 本案例中，VocabDictionary 為抽象介面。
- Client 端與爬蟲端均依賴此介面，符合 DIP。

---

10. 重要 UML 與資料結構符號說明

符號名稱	功能說明
Qualifier	UML 介面關聯中用於表示 Key-Value 關聯的 Key 端詞性標示。
Enum	列舉型態，用於詞性定義 (Part of Speech)。
Map	Key-Value 雜湊結構，用於儲存詞性與定義列表的對應關係。
List	有序集合，用於存放多個定義字串。

---

11. 範例關鍵方法與流程說明

方法/流程	說明
VocabDictionary.lookup(wordSpelling)	傳入單字拼寫，查詢該單字物件，若無則拋出 WordNotExistException。
Adapter.lookup(wordSpelling)	呼叫 Super Vocab Crawler 的 crawl，捕捉 YouSpelledItWrongException，轉成 WordNotExistException。並將回傳的 superworld 轉成 Word。
CommonLanguageInterface.start()	不斷與使用者互動，輸入單字、呼叫 lookup，印出結果或錯誤訊息。

Thetawave.ai

12. 轉接器模式運作比喻：插頭轉接器

- 筆電只有 Type-C 介面，外接裝置只有 USB 插頭。
- 為了能插上筆電，需要一個 **轉接器**，將 USB 轉成 Type-C。
- 轉接器同時實作 Type-C 介面，內部持有 USB 裝置。
- 這樣便解決了兩者介面不相容的問題。

---

13. 轉接器模式的核心真諦

- **保持 Client 端意圖的純粹性與穩定性**，不因底層實作或第三方技術的變動而被迫調整。
- **封裝相容性問題及轉換邏輯於 Adapter 內部**，使整體架構更具彈性與可維護性。
- **促進系統對未來技術更迭的適應力**，降低耦合度。

---

14. 轉接器模式與 SOLID 原則的關聯

原則	內容說明	與轉接器模式的關係
依賴反轉原則 (DIP)	高層模組依賴抽象，不依賴具體實作。	Adapter 讓 Client 依賴抽象介面，底層模組實作該介面。
單一責任原則 (SRP)	類別只負責一件事。	Adapter 專責轉換，不混入其他業務邏輯。
開放封閉原則 (OCP)	對擴展開放，對修改封閉。	Adapter 允許擴展不同介面整合，無需修改 Client。

---

15. 轉接器模式與系統整合的實際意義

- 轉接器模式是整合多方第三方套件時的常見解決方案。
- 透過先定義 Client 端意圖介面，再以 Adapter 封裝第三方實作，減少系統依賴的脆弱性。
- 這種設計能有效降低系統維護成本，確保變動不影響整體架構穩定。

Thetawave.ai

單字查詢系統中核心類別關係表

類別/介面名稱	功能/角色	備註
Word	單字物件，含拼寫與詞性對應的多重定義	Map<PartOfSpeech, List<String>>
PartOfSpeech (Enum)	詞性列舉，定義 8 種詞性	每種詞性有對應顯示符號
VocabDictionary (介面)	抽象查詢介面，定義 lookup(wordSpelling) 方法	Client 端依賴此介面
VocabLookupCommonLanguageInterface	Client 端命令列介面，與使用者互動	依賴 VocabDictionary
SuperVocabCrawler	第三方爬蟲類別，從網路爬取單字定義	不可修改
SuperWorld	爬蟲回傳資料結構，須轉換成 Word	內含 Role (拼寫)與字串定義
SuperVocabCrawlerAdapter	轉接器類別，實作 VocabDictionary，內含 SuperVocabCrawler	負責介面轉換與例外轉換

---

16. 例外轉換流程

原例外類型	轉換後例外類型	轉換位置
YouSpelledItWrongException	WordNotExistException	SuperVocabCrawlerAdapter

- Adapter 必須捕捉第三方例外，轉換成 Client 預期的例外類型，避免 Client 端需理解非預期例外。

Thetawave.ai

17. 單字資料轉換流程

1. Adapter 呼叫 SuperVocabCrawler.crawl(wordSpelling)，取得 SuperWorld 物件。
2. 解析 SuperWorld 中字串，將詞性與定義分離。
3. 重建 Word 物件的 Map<PartOfSpeech, List<String>> 結構。
4. 回傳 Word 給 Client。

---

18. 轉接器模式適用場景

條件	備註
需整合第三方類別或套件	第三方類別不可更改
兩個介面不相容	介面方法、例外、回傳資料型態皆不同
希望維持 Client 端介面穩定	不希望 Client 程式碼因底層改動而受影響
需要封裝轉換邏輯	包含資料格式轉換與例外類型轉換

---

19. 轉接器模式與軟體設計穩定性的關係

- 透過轉接器，系統高層模組依賴抽象，降低對低層實作的耦合。
- 使系統能夠在第三方技術變更時，僅需修改 Adapter，其他模組不受影響。
- 增強系統的可維護性與可擴展性。

---

20. 小結

轉接器模式透過定義一個 Adapter 類別，實作 Client 所定義的介面，並持有不相容的第三方類別物件，委派其功能並做必要的資料與例外轉換，使兩者

達成相容合作。此模式同時符合依賴反轉原則，強化系統架構的彈性與穩定性，是軟體整合與設計中不可或缺的結構型模式。